

cessing, multimicroprocessor systems, image processing, and speech processing. His current activities include designing a partitionable SIMD/MIMD multimicroprocessor system for image processing.

Dr. Siegel is a member of the Association for Computing Machinery. He was General Chairman of the Workshop on Interconnection Networks for Parallel and Distribution Processing, held in April 1980, which was cospon-

sored by the ACM and the IEEE Computer Society. He is currently the Vice Chairman of ACM SIGARCH (Special Interest Group on Computer Architecture), a Vice Chairman of the IEEE Computer Society Technical Committee on Computer Architecture, and an IEEE Computer Society Distinguished Visitor. He is also a member of the Eta Kappa Nu and Sigma Xi honorary societies.

The Reverse-Exchange Interconnection Network

CHUAN-LIN WU, MEMBER, IEEE, AND TSE-YUN FENG, FELLOW, IEEE

Abstract—Properties of the reverse-exchange interconnection network are used to develop a reconfiguration scheme and a two-pass structure for enhancing the efficiency of a class of multistage interconnection networks. Functional relationships among a class of multistage interconnection networks are first derived. According to the functional relationships, we propose a reconfiguration scheme which enables a network to accomplish various interconnection functions of other networks. Then the admissible permutations along with related recursive control algorithms of the reverse-exchange interconnection network are specified through a set of theorems. Using the reverse-exchange property, we also prove that the algorithms actually work. Finally, we prove that arbitrary permutations can be realized in two passes (or $2 \cdot \log_2 N$ switching steps where N is the network size). By taking advantage of Benes network control algorithms, a way to control the two-pass structure is also developed.

Index Terms—Admissible permutations, baseline network, multiple-pass realization, multistage interconnection networks, parallel processing systems, permutation network, reconfiguration, recursive control algorithm, reverse-exchange network, routing techniques.

I. INTRODUCTION

IN many proposed or existing parallel processing architectures, an interconnection network is used to realize various permutations of data between processors or between processors and memory modules [1]–[11]. The interconnection network in such architectures significantly, and sometimes even dominantly, affects the overall cost and performance. However, there are still problems in designing a cost-effective interconnection network. As a continuation of previous work [11], this paper first addresses some critical problems and then provides relevant contributions through the use of a newly configured multistage interconnection network.

The possibility of realizing arbitrary permutations on a

multistage interconnection network is an important topic to consider. It is noted that all the multistage interconnection networks so far considered [1]–[6], [11] cannot realize arbitrary permutations in a single pass. Pease [4] did point out the necessity of the multiple-pass realization on multistage interconnection networks. However, there is no work available on how to control the multistage interconnection networks to accomplish arbitrary permutations in multiple passes. Using single stage networks, some investigators have presented multiple-pass schemes for realizing arbitrary permutations. These include Lang's $O(\sqrt{N})$ shuffle-exchange steps [7], Orcutt's $O(\sqrt{N} \log_2 N)$ steps of Illiac IV [9], Stone's $O((\log_2 N)^2)$ perfect-shuffle steps [10], and Siegel's $O((\log_2 N)^2)$ algorithm [12]. In reviewing these multiple-pass schemes, we feel that $O(\sqrt{N})$ or $O((\log_2 N)^2)$ is still an impractical number when N is large. An interconnection network which can realize arbitrary permutations in many fewer passes with efficient control techniques available should be favorably considered.

Another important topic to consider is the design of an efficient network control mechanism for both the single pass and the multiple passes. The permutation function and the corresponding control information on a permutation network are closely related. However, it is most likely fair to say that there is only a limited understanding of their relationships. Lawrie [5] proposed a control technique named "destination-tag scheme," which relaxes the requirement of understanding the relationships. This destination-tag scheme can route data from the input side of an omega network to the output side according to the binary address of the destination. A homogeneous routing technique which removes the direction restriction of the destination-tag scheme has also been presented [11]. These routing techniques have to take care of the conflict resolution and to decide the route switching element by switching element. In the case when the needed permutation function is known, there is no need to lose time in calculating the route switching element by switching element. Batcher [3] has done

Manuscript received July 18, 1979; revised April 24, 1980. This work was supported by the Rome Air Development Center under Contract F 30602-76-C-0282.

The authors are with the Department of Computer Science, Wright State University, Dayton, OH 45435.

some work on specifying the control information of the flip network according to the permutation function. However, his work is restricted to the flip permutation functions. On the contrary, for the Benes binary network [13], there are well-defined looping algorithms [14], [15] which calculate the control information according to the permutation function. It is interesting to note that with these looping algorithms, the Benes binary network can realize arbitrary permutations, while its hardware facilities are less than twice those of the multistage interconnection network [1]–[6], [11]. So far there is no work which can take advantage of these looping algorithms in realizing arbitrary permutations on a multistage interconnection network in multiple passes. However, Lenfant [8] pointed out that these looping algorithms are both time- and space-consuming. In order to meet the time constraints arising from the use of a Benes binary network as the alignment network of a parallel computer, Lenfant [8] presented a Benes network control algorithm that can calculate the control information according to the name of five frequently used permutation families. Lenfant and Tahé [16] also derived an external control mechanism for the reverse omega network. Lenfant's approach represents an alternative way in searching for an efficient control mechanism. It is worthy of exploiting useful permutation families of an interconnection network and then developing the control mechanism accordingly.

The third important issue is to exploit the possibility of increasing the combinatorial power of a network through reconfiguring that network. The topological equivalence of a class of multistage interconnection networks has been defined previously [11]. However, the topological equivalence is not equal to the functional equivalence. It would be a favored step to define the functional equivalence and explicitly express the functional relationships among the topologically equivalent interconnection networks. These functional relationships can facilitate obtaining the necessary reconfiguration information so that one can use just one topology and accomplish various functions through reconfiguring.

This paper is organized as follows. In Section II we review our mathematical model developed previously [11] and introduce terminologies of the reverse-exchange network which will be used to demonstrate our solutions to these problems described. Section III provides the functional relationships among topologically equivalent multistage interconnection networks. We also address a way of reconfiguring a network to increase the network combinatorial power. The capability and the recursive control algorithm of the reverse-exchange network are exploited in Sections IV and V, respectively. In Section VI we prove that the reverse-exchange network can realize arbitrary permutation in two passes. Both the construction and the routing scheme are provided. The network utilization and limitations are discussed in Section VII.

II. THE REVERSE-EXCHANGE NETWORK

A mathematical model was previously presented to specify various multistage interconnection networks [11]. For a self-contained purpose, we give a brief review here. A multistage interconnection network can be defined by specifying its *configurations* and its *control structure*. We shall discuss the control structure in the next section. By the configuration of

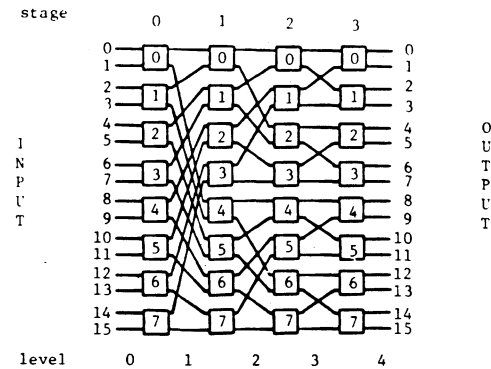
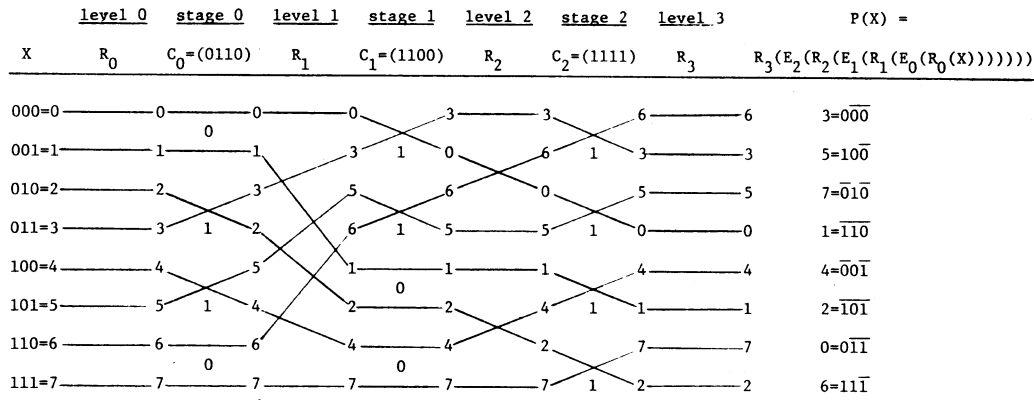


Fig. 1. A configuration of a 16×16 baseline network.

an interconnection network we mean the *topology* and the *logical names* of the network components: interconnection links and switching elements. The topology is defined in terms of three parameters: 1) the number of communication paths of each switching element; 2) the number of switching element stages; and 3) the connectivities of the interconnection links between switching elements. A set of mathematical rules, called *topology describing rules*, is used to describe the connectivities of the interconnection links. Usually the connectivities of an interconnection network are originally defined in terms of the physical position of network components. The logical name which is assigned according to the physical position of network components is also called *physical name*. The topology whose topology describing rules are formulated in terms of the physical name is called *physical topology*. Fig. 1 shows a configuration of a 16×16 baseline network. Note that the names of the interconnection links do not appear in Fig. 1, but they can be derived from the names of the switching elements. With such a configuration, we can uniquely identify every component and unambiguously assign the interconnection task by specifying the input logical name and the output logical name.

The baseline network can be used as a reference to evaluate the relationship among various multistage interconnection networks. A class of topologically equivalent multistage interconnection networks has been defined [11] by using the fact that any network topology in this class can be described by the same topology describing rules of the baseline network if the logical names are properly assigned to the network components. In the rest of this paper, to configure a network means to assign logical names to network components. Note that a network configured in our way may have different functional capabilities from those of the network with the same physical topology, but configured in the other way. We like to exploit some terminologies of this class of multistage interconnection networks before we use it in solving the described problems. Since the networks in the class configured in our way are functionally identical, although they have different physical topologies, we could get the same results no matter which physical topology is used in demonstration. In this paper we use the physical topology of the baseline network.

The permutation function of a network is accomplished by two components: the interconnection links and the switching elements. Assume the binary representation of integer X is $x_l x_{l-1} \dots x_0$ where $l = n - 1$ and $n = \log_2 N$. The link con-

Fig. 2. A permutation realized by the reverse-exchange network of size 2³.

nectivity of level i of the network performs the following permutations:

$$R_i(x_l x_{l-1} \cdots x_0) = x_l \cdots x_{l-i+2} x_0 x_{l-i+1} \cdots x_1 \quad (1)$$

for $0 < i \leq l$. For $i = 0$ and $l + 1$,

$$R_i(x_l x_{l-1} \cdots x_0) = x_l x_{l-1} \cdots x_0. \quad (2)$$

By (1) and (2) we have

$$R_{l+1}(R_l(R_{l-1} \cdots (R_0(x_l x_{l-1} \cdots x_0)) \cdots)) = x_0 x_1 \cdots x_l. \quad (3)$$

Equation (3) implies that the overall interconnection links of the network perform the bit-reverse permutation. The exchange is performed by a switching element on two inputs named by adjacent numbers. The exchange permutation is defined as

$$E(x_l x_{l-1} \cdots x_0) = \begin{cases} x_l x_{l-1} \cdots x_0 & \text{if } c = 0 \\ x_l x_{l-1} \cdots \bar{x}_0 & \text{if } c = 1 \end{cases} \quad (4)$$

where c is the control bit of the switching element. Since there are $N/2$ switching elements in a stage, there is a control vector associated with each stage. The notation of $C_i(j)$ and E_i are used to denote, respectively, the control bit for j th switching element in stage i and the exchange permutation of stage i associated with control vector C_i . Assume that X is permuted to $P(X)$ by the network. Then

$$\begin{aligned} P(x_l x_{l-1} \cdots x_0) &= R_{l+1}(E_l(R_l(E_{l-1} \cdots (R_1(E_0(R_0(x_l x_{l-1} \cdots x_0)))) \cdots)) \\ &= e_0(x_0) \cdot e_1(x_1) \cdots e_{l-1}(x_{l-1}) \cdot e_l(x_l) \end{aligned} \quad (5)$$

where $e_i(x_i)$, $0 \leq i \leq l$ is equal to x_i or $\bar{x}_i$, depending on the exchange performed by the associated switching element in stage i . Fig. 2 shows an example of a permutation realized on the network of size $N = 2^3$ with control vectors as specified. Because of the bit-reverse and the exchange attributes, we use the reverse-exchange network to name this uniquely configured class of multistage interconnection networks.

III. FUNCTIONAL RELATIONSHIPS AND RECONFIGURATION

In this section we compare functional relationships among topologically equivalent multistage interconnection networks [11] and exploit the possibility to reconfigure a network to

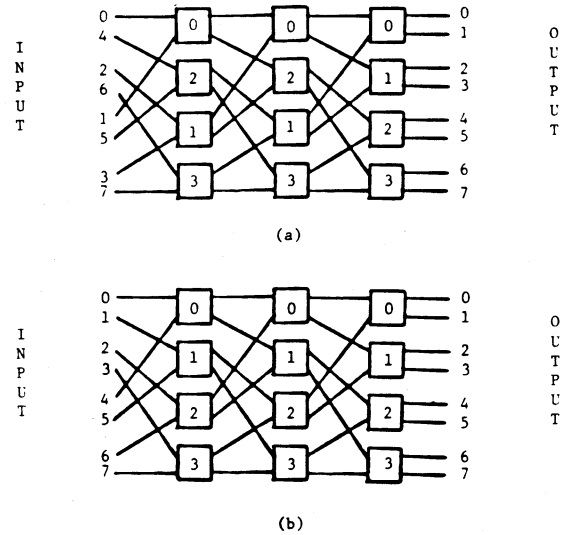


Fig. 3. (a) A reverse-exchange network. (b) An omega network.

accomplish functions of various networks. First of all, we would like to make it clear that the topological equivalence between two networks implies the one-to-one and onto relationship between network components of the two networks, and does not necessarily imply the identical functionality between the two networks. For example, Fig. 3(a) shows an omega network reconfigured by using the *binary tree coding method* [11]. This network is actually a reverse-exchange network and is functionally different from the original omega network proposed by Lawrie [5] as shown in Fig. 3(b). Previously, we used the topology of the baseline network [11] to obtain the topological relationship among various networks. Here, again, we shall use the functions of the baseline network to obtain functional relationships among various networks. But before evaluating the functional relationships, we should define some terminologies and mathematical tools.

The functions of a network are also confined by its control structure. Siegel [17] proposed that the control structure of a network sets the states of a switching element. Thus, the control structure of a network sends the control information to switching elements to realize the interconnection function. We, therefore, propose here that two multistage interconnection networks are *functionally equivalent* (or identical) if

they have a same set of admissible interconnection functions and they can realize any admissible interconnection function using the same control information (without duplication, but a proper mapping on its location is allowed). Hence, in order to compare the functional capability of various networks, we have to assume that those networks have the same control structure which includes the states of each switching element and the way to set them. In this section the functional relationships among various networks are assessed under this assumption. Under the assumption, let the set of admissible network functions of the baseline network, the reverse baseline network, the omega network, the reverse omega network, the indirect binary n -cube network, the reverse indirect binary n -cube network, the modified data manipulator, the reverse modified data manipulator, the flip network, the reverse flip network, the regular SW banyan network ($S = F = 2$), and the reverse regular SW banyan network ($S = F = 2$) be B_N , B_N^{-1} , Ω_N , Ω_N^{-1} , C_N , C_N^{-1} , D_N , D_N^{-1} , F_N , F_N^{-1} , G_N , and G_N^{-1} , respectively. Using the previous notations of the binary representation, we define the *bit reversal permutation* ρ by

$$\rho(x_l x_{l-1} \cdots x_0) = x_0 x_1 \cdots x_l \quad (6)$$

and the *bit switch permutation* δ by

$$\delta(x_l x_{l-1} x_{l-2} \cdots x_3 x_2 x_1 x_0) = x_1 x_2 x_3 \cdots x_{l-2} x_{l-1} x_l x_0 \quad (7)$$

for $l \geq 1$.

Using the above definitions and the binary tree coding method, we can now evaluate the functional relationships. A simple example will be demonstrated on the networks shown in Fig. 3. The reverse-exchange network shown in Fig. 3(a) is configured by using the binary tree coding method and has B_N as its set of admissible network functions where $N = 8$. The omega network shown in Fig. 3(b), of course, has Ω_N as its set of admissible network functions where $N = 8$. Now if we take a bit-reverse permutation ρ on the logical names of the input links of the omega network shown in Fig. 3(b), we should have the same logical names for the input links as those shown in Fig. 3(a). Therefore, from this example we can see, for $N = 8$,

$$\Omega_N = B_N \circ \rho, \quad (8)$$

where $B_N \circ \rho$ implies $B_N(\rho)$. Equation (8) means that the omega network function can be decomposed into two ordered functions: first, a bit-reverse permutation, and then, a baseline network function. The argument can be extended for any N . The functional relationships between the baseline network and other topological equivalent networks can be similarly developed. The result is listed in the $\theta = B_N$ column of Table I. Note that the bit switch permutation δ is used in the D_N , D_N^{-1} , G_N , and G_N^{-1} rows. It is interesting to see that the baseline network and the reverse baseline network are functionally identical as indicated in Table I by

$$B_N^{-1} = B_N, \quad (9)$$

The above results in which we use the B_N as the reference can be manipulated so that new formulas can be formed in terms

TABLE I
FUNCTIONAL RELATIONSHIPS AMONG VARIOUS MULTISTAGE
INTERCONNECTION NETWORKS ($P = f(\theta)$).

Network Function (P)	$f(\theta)$				
	$\theta = B_N$	$\theta = \Omega_N$	$\theta = C_N$	$\theta = D_N$	$\theta = G_N$
Baseline Network (B_N)	B_N	$\Omega_N \circ \rho$	$\rho \circ C_N$	$D_N \circ \delta$	$\delta \circ G_N$
Reverse Baseline Network (B_N^{-1})	B_N	$\Omega_N \circ \rho$	$\rho \circ C_N$	$D_N \circ \delta$	$\delta \circ G_N$
Omega Network (Ω_N)	$B_N \circ \rho$	Ω_N	$\rho \circ C_N \circ \rho$	$D_N \circ \delta \circ \rho$	$\delta \circ G_N \circ \rho$
Reverse Omega Network (Ω_N^{-1})	$\rho \circ B_N$	$\rho \circ \Omega_N \circ \rho$	C_N	$\rho \circ D_N \circ \delta$	$\rho \circ \delta \circ G_N$
Indirect Binary n -Cube Network (C_N)	$\rho \circ B_N$	$\rho \circ \Omega_N \circ \rho$	C_N	$\rho \circ D_N \circ \delta$	$\rho \circ \delta \circ G_N$
Reverse Indirect Binary n -Cube Network (C_N^{-1})	$B_N \circ \rho$	Ω_N	$\rho \circ C_N \circ \rho$	$D_N \circ \delta \circ \rho$	$\delta \circ G_N \circ \rho$
Modified Data Manipulator (D_N)	$B_N \circ \delta$	$\Omega_N \circ \rho \circ \delta$	$\rho \circ C_N \circ \delta$	D_N	$\delta \circ G_N \circ \delta$
Reverse Modified Data Manipulator (D_N^{-1})	$\delta \circ B_N$	$\delta \circ \Omega_N \circ \rho$	$\delta \circ \rho \circ C_N$	$\delta \circ D_N \circ \delta$	G_N
Flip Network (F_N)	$\rho \circ B_N$	$\rho \circ \Omega_N \circ \rho$	C_N	$\rho \circ D_N \circ \delta$	$\rho \circ \delta \circ G_N$
Reverse Flip Network (F_N^{-1})	$B_N \circ \rho$	Ω_N	$\rho \circ C_N \circ \rho$	$D_N \circ \delta \circ \rho$	$\delta \circ G_N \circ \rho$
Regular SW Banyan Network (G_N) ($S=F=2$)	$\delta \circ B_N$	$\delta \circ \Omega_N \circ \rho$	$\delta \circ \rho \circ C_N$	$\delta \circ D_N \circ \delta$	G_N
Reverse Regular SW Banyan Network (G_N^{-1}) ($S=F=2$)	$B_N \circ \delta$	$\Omega_N \circ \rho \circ \delta$	$\rho \circ C_N \circ \delta$	D_N	$\delta \circ G_N \circ \delta$

of other network functions. For example, we can see that

$$B_N = B_N \circ \rho \circ \rho, \quad (10)$$

since $\rho \circ \rho$ is equal to the identity. Then according to (8) and (10) we have

$$B_N = \Omega_N \circ \rho. \quad (11)$$

This result is shown in the first entry in the $\theta = \Omega_N$ column of Table I. The other entries of Table I are obtained in the same way. From the $\theta = \Omega_N$ column of Table I we can see that $\Omega_N = C_N^{-1} = F_N^{-1}$ which means that the omega network, the reverse indirect binary n -cube network, and the reverse flip network are functionally identical. Similarly, from other columns, we can also see that $C_N = \Omega_N^{-1} = F_N$, $D_N = G_N^{-1}$, and $G_N = D_N^{-1}$.

The functional relationships shown in Table I lay the groundwork for the reconfiguration problem. The reconfiguration problem can be defined as the problem to reassign the logical names of the input links and/or the output links of an interconnection network in a parallel processing system so that the interconnection network can at least realize various functions of the networks listed in Table I. For a simple illustration assume that the baseline network of size N is installed

in a parallel processing system. In order to reconfigure the baseline network to accomplish various functions listed in Table I, two permutation functions ρ and σ are needed to reassign the logical names of terminal links. Assume that we want to realize Ω_N on the topology of the baseline network. Then by (8) the logical name of the input link should be $\rho(x_l x_{l-1} \cdots x_0)$ where $x_l x_{l-1} \cdots x_0$ is the original logical name of the input link in the baseline network, and the logical name of the output link should remain the same. On the other hand, if we want to realize G_N , then by the following formula,

$$G_N = \delta \circ B_N, \quad (12)$$

the logical name of the input link should remain the same, and the logical name of the output link should be $\delta(x_l x_{l-1} \cdots x_0)$. In concluding the way to reconfigure a reference network from (8) and (12), we see that in Table I the permutation appearing on the right-hand side of the reference network function should be used to assign the logical name of the input link, and the permutation appearing on the left-hand side should be used to assign the logical name of the output link.

Incorporating a reconfiguration function in a parallel processing system is highly recommended. The reconfiguration mechanisms should choose a proper network configuration according to the algorithm to be processed. It seems necessary to have a table which establishes the mapping between algorithms and various configurations. Furthermore, some hardware facilities may be needed for this reconfiguration mechanism. Some consequent questions may arise from the use of the reconfiguration. How many configurations on a network topology are necessary to accomplish arbitrary permutations? What is the relationship between the configuration and the frequently used permutations [8]? Does the reconfiguration have any impact on the multidimensional-access scheme [5], [19]? We will discuss these questions in Section VII on the network utilization and limitations.

IV. ADMISSIBLE PERMUTATIONS

The admissible permutations, i.e., permutations which can be realized by the reverse-exchange network in one pass, are a subset of $2^n!$ distinct permutations. In this section we will identify these admissible permutations.

Assume that $(A_j, Z_j) = (a_j, l a_{j,l-1} \cdots a_{j,0}, z_j, l z_{j,l-1} \cdots z_{j,0})$ represents the source-destination pair of the permutation request $P(A_j) = Z_j$. Define $A_{p,q} = a_{p,l} a_{p,l-1} \cdots a_{p,l-q+1}$ and $Z_{p,q} = z_{p,l} z_{p,l-1} \cdots z_{p,l-q+1}$.

Theorem 1: Given a set of distinct permutation requests $P_N = \{(A_i, Z_i) | 0 \leq i < N\}$, P_N can be realized by the reverse-exchange network if and only if $A_j \neq A_k$ and $A_{j,l-m} = A_{k,l-m}$ implies $Z_{j,m+1} \neq Z_{k,m+1}$ for $j \neq k$, $0 \leq j, k < N$, and $0 \leq m \leq l$.

Proof: Recall from previous work [11] that for the permutation request $P(A_j) = Z_j$, stage m will switch source $a_{j,l} a_{j,l-1} \cdots a_{j,0}$ to link $z_{j,l} z_{j,l-1} \cdots z_{j,l-m+1} a_{j,l} a_{j,l-1} \cdots a_{j,m+1} z_{j,l-m}$ in level $m+1$. A conflict occurs if some other source is also switched to this link. That is, for some pairs of permutation requests, say (A_j, Z_j) and (A_k, Z_k) , and for some m , we have $a_{j,l} a_{j,l-1} \cdots a_{j,m+1} = a_{k,l} a_{k,l-1} \cdots a_{k,m+1}$ and

$z_{j,l} z_{j,l-1} \cdots z_{j,l-m} = z_{k,l} z_{k,l-1} \cdots z_{k,l-m}$. The conflict condition can be represented as $A_{j,l-m} = A_{k,l-m}$ and $Z_{j,m+1} = Z_{k,m+1}$ for permutation requests $P(A_j) = Z_j$ and $P(A_k) = Z_k$. Therefore, if $A_j \neq A_k$ and $A_{j,l-m} = A_{k,l-m}$ implies $Z_{j,m+1} \neq Z_{k,m+1}$ for $j \neq k$, $0 \leq j, k < N$ and $0 \leq m \leq l$, both (A_j, Z_j) and (A_k, Z_k) are realizable permutations. On the other hand, since there is only one interconnection path existing for a permutation, the reverse way of the above statement is also true. Q.E.D.

Note that if we take (11) into account, Theorem 1 can also be inferred from the similar arguments of Lawrie for Ω_N [5]. Using Theorem 1, we will identify some of the permutations which can be realized by the reverse-exchange network in one pass.

Theorem 2: Define X'_i to be the number whose binary representation is the reverse binary representation of X_i and define $P_N = \{(X_i, X'_i) | 0 \leq i < N\}$ to be the bit-reverse permutation. Then P_N is realizable by the reverse-exchange network.

Proof: Assume that $X_{j,l-t} = X_{k,l-t}$ for $0 \leq t \leq l-1$ where $j \neq k$. Since $X_j \neq X_k$ for $j \neq k$, we then obtain $X'_{j,t+1} \neq X'_{k,t+1}$ from the assumption. The proof immediately follows Theorem 1. Q.E.D.

Theorem 3: If $P_N = \{(A_i, Z_i) | 0 \leq i < N\}$ is realizable by the reverse-exchange network and a is an odd integer, then $P'_N = \{(A_i, aZ_i) | 0 \leq i < N\}$ is also realizable by the network.

Proof: Define $Y_i = aZ_i$. We will prove that $A_{j,l-m} = A_{k,l-m}$ for $j \neq k$ and $0 \leq m \leq l$ implies that $Y_{j,m+1} \neq Y_{k,m+1}$.

Since $A_{j,l-m} = A_{k,l-m}$ for $j \neq k$ and $0 \leq m \leq l$ implies that $Z_{j,m+1} \neq Z_{k,m+1}$, hence, $a_{j,l} a_{j,l-1} \cdots a_{j,m+1} = a_{k,l} a_{k,l-1} \cdots a_{k,m+1}$ implies that $z_{j,m} z_{j,m-1} \cdots z_{j,0} \neq z_{k,m} z_{k,m-1} \cdots z_{k,0}$. Assume further that $z_{j,t} \neq z_{k,t}$ for some t , $0 \leq t \leq m$ and $z_{j,s} = z_{k,s}$ for $t < s \leq m$. Let $b_l b_{l-1} \cdots b_0$ be the binary representation of a where $b_0 = 1$ since a is odd. The products of $Y_j = aZ_j$ and $Y_k = aZ_k$ will result in $y_{j,t} \neq y_{k,t}$. This result concludes that $A_{j,l-m} = A_{k,l-m}$ for $j \neq k$ and $0 \leq m < l$ implies that $Y_{j,m+1} \neq Y_{k,m+1}$. Q.E.D.

Theorem 4: If $P_N = \{(A_i, Z_i) | 0 \leq i < N\}$ is realizable by the reverse-exchange network and b is an integer, then $P_N = \{(A_i, Z_i + b) | 0 \leq i < N\}$ is also realizable by the network.

Proof: Let $Y_i = Z_i + b$. It is obvious that $Z_{j,m+1} \neq Z_{k,m+1}$ for $0 \leq m < l$ implies that $Y_{j,m+1} \neq Y_{k,m+1}$. Hence, $A_{j,l-m} = A_{k,l-m}$ for $j \neq k$ and $0 \leq m < l$ implies that $Z_{j,m+1} \neq Z_{k,m+1}$ and also $Y_{j,m+1} \neq Y_{k,m+1}$. Q.E.D.

Corollary 1: The permutation defined by $P_N = \{(X_i, aX'_i + b) | 0 \leq i < N, a \text{ is an odd integer and } b \text{ is an integer}\}$ is realizable by the reverse-exchange network.

Corollary 2: The permutation defined by $P_N = \{(X_i, T - X'_i) | 0 \leq i < N \text{ and } T \text{ is an integer}\}$ is realizable by the reverse-exchange network.

Corollaries 1 and 2 are consequences of Theorems 2, 3, and 4. Note that the omega network has the same properties shown by Theorems 3 and 4 [5].

Theorem 5: If $P_N = \{(A_i, Z_i) | 0 \leq i < N\}$ is realizable by the reverse-exchange network and k is an integer, then $P'_N = \{(A_i, Z_i \oplus k) | 0 \leq i < N \text{ and } \oplus \text{ is the bit-by-bit EXCLUSIVE OR}\}$ is also realizable by the network.

Proof: Define $Y_i = Z_i \oplus k$. It can be seen that $Y_{j,m} \neq$

$Y_{k,m}$ if $Z_{j,m} \neq Z_{k,m}$ for $0 \leq m \leq l$. Since P_N is realizable by the reverse-exchange network, $A_{j,l-m} = A_{k,l-m}$ for $j \neq k$ and $0 \leq m < l$ also implies that $Y_{j,m+1} \neq Y_{k,m+1}$. Q.E.D.

Corollary 3: The permutation defined by $P_N = \{(X_i, X_i' \oplus k) | 0 \leq i < N \text{ and } k \text{ is an integer}\}$ is realizable by the reverse-exchange network.

Corollary 3 is a consequence of Theorems 2 and 5.

Theorem 6: Define the following binary representations.

$$X_i = (x_{i,l}x_{i,l-1} \cdots x_{i,j}x_{i,j-1} \cdots x_{i,0}),$$

$$U = (x_{i,j-1} \cdots x_{i,0}),$$

$$Y_i = (v_{i,0} \cdots v_{i,j-1}x_{i,j} \cdots x_{k,l-1}x_{i,l}),$$

and

$$V = (v_{i,j-1}v_{i,j-2} \cdots v_{i,0}).$$

Assume that $V = U + k \bmod 2^j$ where k is an integer. Then the permutation defined by $P_N = \{(X_i, Y_i) | 0 \leq i < N\}$ is realizable by the reverse-exchange network.

Proof: By the definition, if $(x_{p,m}x_{p,m-1} \cdots x_{p,0}) \neq (x_{q,m}x_{q,m-1} \cdots x_{q,0})$, then $(y_{p,l}y_{p,l-1} \cdots y_{p,l-m}) \neq (y_{q,l}y_{q,l-1} \cdots y_{q,l-m})$ for either $m \leq j$ or $m > j$. Hence, $X_{p,l-m} = X_{q,l-m}$ for $p \neq q$ and $0 \leq m < l$ implies that $Y_{p,m+1} \neq Y_{q,m+1}$. Q.E.D.

Theorem 7: If $P_N = \{(A_i, Z_i) | 0 \leq i < N\}$ is realizable by the reverse-exchange network, then $P'_N = \{(Z_i, A_i) | 0 \leq i < N\}$ is also realizable by the network.

Proof: Since P_N is realizable by the reverse-exchange network, $a_{j,l} \cdots a_{j,m+1} = a_{k,l} \cdots a_{k,m+1}$ for $j \neq k$ and $0 \leq m < l$ implies that $z_{j,l} \cdots z_{j,l-m} \neq z_{k,l} \cdots z_{k,l-m}$. By contradiction, assume that in the case of $z_{j,l} \cdots z_{j,q+1} = z_{k,l} \cdots z_{k,q+1}$ for $j \neq k$ and $0 \leq q < l$, we can have $a_{j,l} \cdots a_{j,l-q} = a_{k,l} \cdots a_{k,l-q}$. Then there exists $m = l - q - 1$ such that $a_{j,l} \cdots a_{j,m+1} = a_{k,l} \cdots a_{k,m+1}$ and $z_{j,l} \cdots z_{j,l-m} = z_{k,l} \cdots z_{k,l-m}$. This contradicts the statement shown in the beginning of this proof. Hence, $z_{j,l} \cdots z_{j,q+1} = z_{k,l} \cdots z_{k,q+1}$ for $j \neq k$ and $0 \leq q < l$ implies that $a_{j,l} \cdots a_{j,l-q} \neq a_{k,l} \cdots a_{k,l-q}$. Q.E.D.

Corollary 4: The permutation defined by $P_N = \{(aX_i + b, X_i') | 0 \leq i < N, a \text{ is an odd integer and } b \text{ is an integer}\}$ is realizable by the reverse-exchange network.

Proof: According to Theorems 2 and 7, we can see that $P'_N = \{(X_i', X_i) | 0 \leq i < N\}$ is realizable by the network. Then by Theorems 3 and 4 it is obvious that $P''_N = \{(X_i', aX_i + b) | 0 \leq i < N, a \text{ is an odd integer and } b \text{ is an integer}\}$ is realizable. From Theorem 7 again, we see that $P_N = \{(aX_i + b, X_i') | 0 \leq i < N, a \text{ is an odd integer and } b \text{ is an integer}\}$ is realizable. Q.E.D.

We have presented some theorems which identify the one-pass realizable permutations of the reverse-exchange network. In the next section we will use these theorems to classify some especially interesting realizable permutations and exploit the relationships between the permutation and the corresponding control information.

V. RECURSIVE ROUTING MECHANISM FOR ADMISSIBLE PERMUTATIONS

The homogeneous routing procedure described previously in [11] already provides a simple routing mechanism. It em-

plays the n -bit destination tag to determine the valid state of the in-path switching elements. However, because of the prescribed reasons, it is preferable to determine the control pattern by the name of the admissible permutation. We will first identify some classes of useful admissible permutations and then derive recursive algorithms which determine control patterns according to the permutation names.

It is easy to verify by Theorems 1-7 that the following categories of important permutations are admissible on the reverse-exchange network.

- 1) $F_k^{(n)}(0 \leq k < 2^n)$: $P(X^r \oplus k) = X$ and $P(X \oplus k) = X^r$.
- 2) $C_{j,k}^{(n)}(0 \leq j, k < 2^n, j \text{ odd})$: $P(jX^r + k) = X$ and $P(jX + k) = X^r$.
- 3) $S_{q,k}^{(n)}(0 < q \leq n, 0 < k < 2^q)$: $P(X) = (X + k - \epsilon \cdot 2^q)^r$ and $P(X + 2^q - k - \epsilon \cdot 2^q) = X^r$.
- 4) $\tilde{F}_k^{(n)}(0 \leq k < 2^n)$: $P(X) = X^r \oplus k$ and $P(X^r) = X \oplus k$.
- 5) $\tilde{C}_{j,k}^{(n)}(0 \leq j, k < 2^n, j \text{ odd})$: $P(X) = jX^r + k$ and $P(X^r) = jX + k$.
- 6) $\tilde{S}_{q,k}^{(n)}(0 < q \leq n, 0 < k < 2^q)$: $P((X + k - \epsilon \cdot 2^q)^r) = X$ and $P(X^r) = X + 2^q - k - \epsilon \cdot 2^q$.

Note that Category 4) [5] and 6)] is a consequence of applying Theorem 7 to Category 1) [2] and 3)]. Also, $S_{q,k}^{(n)}$ of Category 3), a direct result of Theorem 6, is a cyclic shift of amplitude k within each segment of size 2^q . The ϵ in $S_{q,k}^{(n)}$ can be defined as follows. Let $X = x_lx_{l-1} \cdots x_qx_{q-1} \cdots x_0$ and $k = 0 \cdots 0k_{q-1} \cdots k_0$. Then ϵ is equal to the carry out bit of summing $x_{q-1}x_{q-2} \cdots x_0$ and $k_{q-1}k_{q-2} \cdots k_0$.

Define $K^{(n)}(P) = (v_{ij})$, $0 \leq i < 2^{n-1}$ and $0 \leq j \leq n-1$ as the control pattern of a $2^{n-1} \times n$ matrix associated with a permutation P . Define also $V^{(n-j)}(b)$ as the 2^{n-j} bit vector whose components are all equal to b . A binary tree whose root is a vector V and whose upper subtree and lower subtree are K_u and K_v , respectively, is denoted by $[V; K_u, K_v]$. The cascaded matrix whose left part and right part are L and R , respectively, is denoted by $[L; R]$.

Let k by a positive integer and denote, respectively, by k' and k_0 its quotient and its remainder in the division by 2, i.e., $k = 2k' + k_0$.

Theorem 8: $K^{(n)}(F_k^{(n)}) = [V^{(n-1)}(k_0); K^{(n-1)}(F_{k'}^{(n)})]$ where $K^{(1)}(F_{k'}^{(n)}) = [V^{(n-1)}(k')]$.

Proof: Let $X = x_lx_{l-1} \cdots x_0$ and $k = k_lk_{l-1} \cdots k_0$ where $l = n-1$. Then $X^r \oplus k = (x_0 \oplus k_l)(x_1 \oplus k_{l-1}) \cdots (x_l \oplus k_0)$. According to the properties of the reverse-exchange network described by (5), we have $x_i = e_{l-i}(x_i \oplus k_{l-i})$, $0 \leq i \leq l$ for $P(X^r \oplus k) = X$. Therefore, we can see that, for $0 \leq i \leq l$,

$$x_i = \begin{cases} e_{l-i}(x_i) & \text{if } k_{l-i} = 0, \\ e_{l-i}(\bar{x}_i) & \text{if } k_{l-i} = 1. \end{cases} \quad (13)$$

Equation (13), in turn, implies that the control bit of switching elements in stage $l-i$ should equal to k_{l-i} . Thus, we obtain

$$K^n(F_k^{(n)}) = [V^{(n-1)}(k_0); V^{(n-1)}(k_1); \cdots; V^{(n-1)}(k_l)]. \quad (14)$$

Equation (14) is exactly what the theorem implies. The same arguments can be made for $P(X \oplus k) = X^r$. Q.E.D.

Example: Assume that

$$P = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 5 & 3 & 7 & 0 & 4 & 2 & 6 \end{pmatrix}.$$

P can be described by $F_4^{(3)}$: $P(X^r \oplus 4) = X$. According to Theorem 8, we have $K^{(3)}(F_4^{(3)}) = [v^{(2)}(0); v^{(2)}(0); v^{(2)}(1)]$. Hence,

$$K^{(3)}(P) = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

The setting of the network is depicted in Fig. 4.

Theorem 9: Let $j = 2j' + 1$. $K^{(n)}(C_{j,k}^{(n)}) = [V^{(n-1)}(k_0); K^{(n-1)}(C_{j,k'}^{(n-1)}), K^{(n-1)}(C_{j,j'+k'+k_0}^{(n-1)})]$ where $K^{(1)}(C_{j,k}^{(1)}) = [k]$.

Proof: Consider $P(jX^r + k) = X$. We first define some notations.

$$X = x_l x_{l-1} \cdots x_0,$$

$$X' = x_l x_{l-1} \cdots x_1,$$

$$j = j_l j_{l-1} \cdots j_0,$$

$$j' = j_l j_{l-1} \cdots j_1,$$

$$k = k_l k_{l-1} \cdots k_0,$$

$$k' = k_l k_{l-1} \cdots k_1,$$

$$Y = jX^r + k = y_l y_{l-1} \cdots y_0,$$

and

$$Y' = y_l y_{l-1} \cdots y_1.$$

Note that $X = 2X' + x_0$, $Y = 2Y' + y_0$, $j = 2j' + 1$, and $k = 2k' + k_0$. Furthermore, $y_0 = x_l \oplus k_0$ and $Y' = j(X')^r + j'x_l + k' + x_l k_0$.

According to (5), we have

$$e_0(y_0)e_1(y_1) \cdots e_l(y_l) = x_l x_{l-1} \cdots x_0. \quad (15)$$

Hence, by (15) the control bit of switching elements in stage 0 is determined by $e_0(y_0) = x_l$. Since $y_0 = x_l \oplus k_0$, the control vector of stage 0 is then equal to $V^{(n-1)}(k_0)$. The next $n-1$ control vectors are dependent on $Y' = y_l y_{l-1} \cdots y_1$. These control vectors can then be partitioned into two halves: the upper half where $x_l = 0$, and the lower half where $x_l = 1$. For the upper half, $Y' = j(X')^r + k'$ since $x_l = 0$. The permutation for this upper half then becomes $P(j(X')^r + k') = X'$. For the lower half, $Y' = j(X')^r + j' + k' + k_0$ since $x_l = 1$. The permutation for this lower half then becomes $P(j(X')^r + j' + k' + k_0) = X'$. The above discussion can recursively be applied to these two permutations of the upper half and the lower half, as shown in the theorem. Similar arguments can be made for $P(jX + k) = X^r$. Q.E.D.

Example: Assume that

$$P = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 \\ 10 & 4 & 15 & 3 & 9 & 6 & 12 & 0 & 11 & 5 & 14 & 2 & 8 & 7 & 13 & 1 \end{pmatrix}.$$

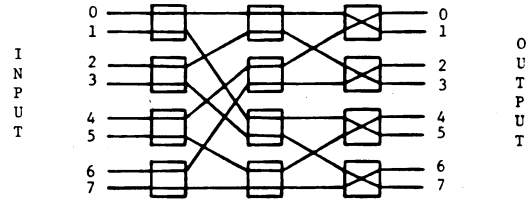


Fig. 4. Setting for $F_4^{(3)}$.

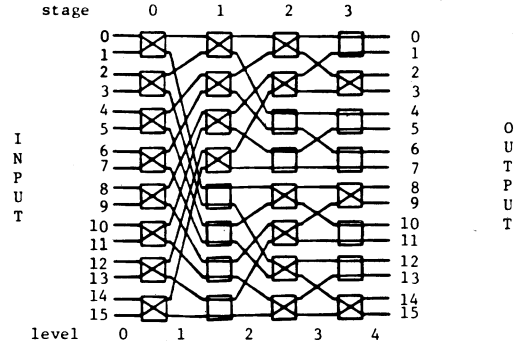


Fig. 5. Setting for $C_{5,7}^{(4)}$.

P can be described by $C_{5,7}^{(4)}$: $P(5X^r + 7) = X$. According to Theorem 9, we have

$$K^{(4)}(C_{5,7}^{(4)}) = \begin{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} & \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \end{bmatrix}$$

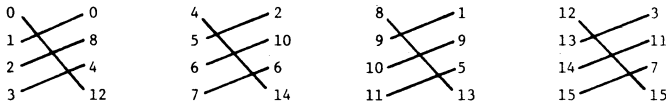
The setting of the network is depicted in Fig. 5.

Theorem 10: For $q \leq n$ and $0 < k < 2^q$, $K^{(n)}(S_{q,k}^{(n)}) = [V^{(n-1)}(k_0); K^{(n-1)}(S_{q,k'+k_0}^{(n-1)}), K^{(n-1)}(S_{q,k'}^{(n-1)})]$, and $K^{(n-q)}(S_{q,k}^{(n-q)}) = K^{(n-q)}(C_{1,0}^{(n-q)})$. If $q = k$, then $K^{(1)}(S_{q,k}^{(1)}) = [k_0]$.

Proof: Consider $P(X) = (X + k - \epsilon \cdot 2^q)^r$ here. According to the value of ϵ defined previously, we can see that the permutation is equivalent to $P(X) = (X + k)^r$ for stage i where $i < q$ and $P(X) = X^r$ for stage j where $j \geq q$. Let ξ_i be the carry out of the sum of x_m and k_m for $0 \leq m < i$ where $\xi_0 = 0$. For stage i , $i < q$, we have

$$e_i(x_i) = x_i \oplus k_i \oplus \xi_i. \quad (16)$$

From (16) we have $e_0(x_0) = x_0 \oplus k_0$ which implies $V^{(n-1)}(k_0)$ for stage 0. Similarly to the proof of Theorem 9, we partition the next $(n-1)$ control vectors into two halves: the upper half where $x_0 \oplus k_0 = 0$ and the lower half where $x_0 \oplus k_0 = 1$. In the upper half, if $k_0 = 1$, then $x_0 = 1$, and if $k_0 = 0$, then $x_0 = 0$. Therefore, $\xi_1 = k_0$ for the upper half. Thus, the control vectors of the upper half can be described by $K^{(n-1)}(S_{q,k'+k_0}^{(n-1)})$. Since $\xi_1 = 0$ for the lower half, its control vector can be described by $K^{(n-1)}(S_{q,k'}^{(n-1)})$. Concluding the above arguments for stage i where $i < q$, we have $K^{(n)}(S_{q,k}^{(n)}) = [V^{(n-1)}(k_0); K^{(n-1)}(S_{q,k'+k_0}^{(n-1)}), K^{(n-1)}(S_{q,k'}^{(n-1)})]$. On the other hand, for stage

Fig. 6. $S_{2,3}^{(4)}$.

j where $j \geq q$, we can obtain the control vectors from Theorem 9 since $P(X) = X^r$ is a case considered in Theorem 9 where $j = 1$ and $k = 0$. Thus, $K^{(n-q)}(S_{q,k}^{(n-q)}) = K^{(n-q)}(C_{1,0}^{(n-q)})$. Similar arguments can be made for $P(X + 2^q - k - \epsilon \cdot 2^q) = X^r$. Q.E.D.

Example: Assume that

$$P = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 \\ 12 & 0 & 8 & 4 & 14 & 2 & 10 & 6 & 13 & 1 & 9 & 5 & 15 & 3 & 11 & 7 \end{pmatrix}.$$

P can be described by $S_{2,3}^{(4)}$ which is a cyclic shift of amplitude 3 within each segment of size 2^2 as shown in Fig. 6. The application of the algorithm shown in Theorem 10 results in

$$K^{(4)}(S_{2,3}^{(4)}) = \begin{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{bmatrix}.$$

The setting of the network is depicted in Fig. 7.

Corollary 5: The control bit pattern of Category 4) [5) and 6)] can be obtained by reversing and properly permuting that of Category 1) [2) and 3)].

Proof: Since the baseline network and the reverse baseline network are topologically and functionally equivalent, the reverse and the permutation of the control bit location are obvious from one to another. The rule for permuting the control bit is defined in [11] and is recited here:

$$r_i[(P_l P_{l-1} \cdots P_1)_i] = (P_i \cdots P_1 P \cdots P_{i+1})_i \quad \text{for } 0 \leq i \leq l. \quad (17)$$

In (17), $(P_l P_{l-1} \cdots P_1)_i$ is the position of a control bit in stage i after the stage order is reversed, and $(P_i \cdots P_1 P \cdots P_{i+1})_i$ is the correct bit position after permuting. Q.E.D.

VI. REALIZATION OF ARBITRARY PERMUTATIONS

In this section we will first show that arbitrary permutations can be realized by the reverse-exchange network in two passes. Next, we consider the routing scheme for this two-pass structure.

A. Two-pass Permutation

The fact that the Benes binary network can realize arbitrary permutations between its inputs and outputs follows the result of the Slepian-Dupud theorem [13]. Using the above fact, we will prove that the reverse-exchange network can realize arbitrary permutations in two passes.

Theorem 11: The reverse-exchange network can realize arbitrary permutations in two passes.

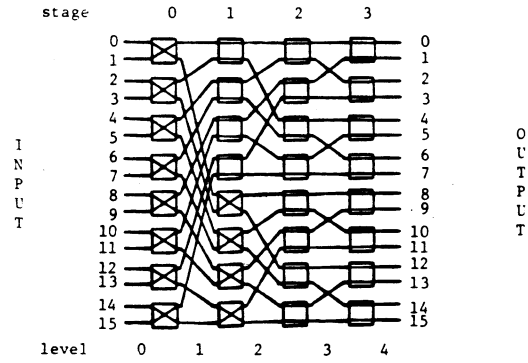
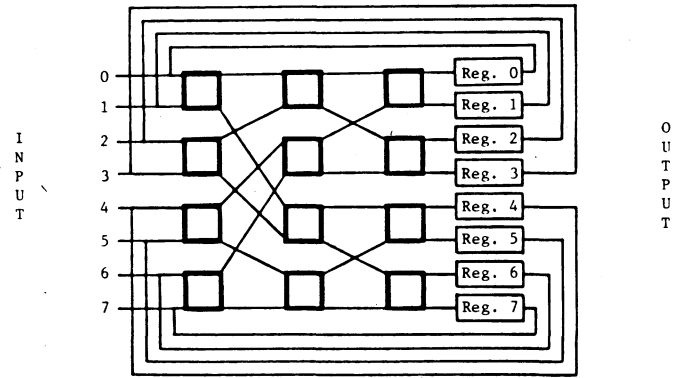
Fig. 7. Setting for $S_{2,3}^{(4)}$.

Fig. 8. Two-pass structure of a reverse-exchange network.

Proof: The theorem will be proven by showing that the functions of the Benes binary network can be simulated by the baseline network in two passes. An example structure for a two-pass implementation using a baseline network is shown in Fig. 8. The input data are fed in on the input side. The output data of the first pass are stored in the shift register files on the output side. In the second pass, the data in the register files are fed back to the input lines and the final results are again stored in the register files.

The two-pass structure is equivalent to the implementations of cascading two baseline networks. According to the previous result, we can obtain a reverse baseline network via properly permuting the switching elements and its related links of the baseline network. For switching elements in the reverse baseline network, the mapping γ_i from physical names $(p_l p_{l-1} \cdots p_1)_i$ to logical names $(b_l b_{l-1} \cdots b_1)_i$ is shown in (17). The mapping λ_i^{-1} from logical names to physical names is shown in the following:

$$\gamma_i^{-1}[(b_l b_{l-1} \cdots b_1)_i] = (b_{l-i} \cdots b_1 b_l \cdots b_{l-i+1})_i \quad (18)$$

for $0 \leq i \leq l$. If we rearrange the switching elements and its related links of the baseline network for the second pass in the equivalent structure in ascending order of the physical names, which are obtained by applying γ_i^{-1} of (18) on the logical names, we can obtain a structure which is formed by a baseline network and a reverse baseline network connected end to front. An example is shown in Fig. 9. The labelings shown in Fig. 9 are the logical names. Now, set the switching elements in the first stage of the reverse baseline network in the equivalent

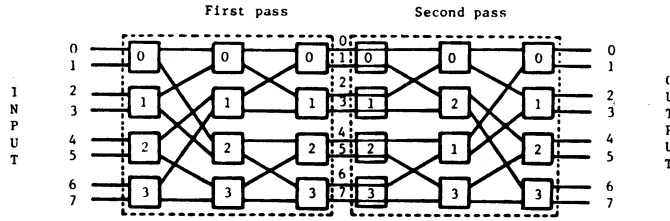


Fig. 9. Equivalent structure and switching element confinement of a two-pass structure.

structure on the state of the direct connection. The setting is also shown in Fig. 9 for the example. It can be seen that the structure shown in Fig. 9 is equivalent to a Benes binary network. Q.E.D.

B. Routing Scheme

Several algorithms exist which compute the control information for the Benes binary network. A simple algorithm was previously shown in [18]. However, more systematic algorithms have been proposed by Opferman and Tsao-Wu [14], Anderson [15], and Lenfant [8]. We shall use these results to derive the control information for our two-pass structure.

The control pattern computed by these algorithms should be properly permuted before it can be applied to the reverse-exchange network. As shown in Theorem 11, the leftmost n stages of the Benes binary network are one-to-one correspondent to the reverse-exchange network of the first pass from left to right, and the rightmost $n - 1$ stages of the Benes binary network are one-to-one correspondent to the reverse-exchange network of the second pass from right to left. The switching elements in the leftmost stage of the network in the second pass are confined in the valid state of the direct connection. Assume that signals 0 and 1 represent the valid states for the direct and the crossed connection, respectively. We can represent the control patterns of the Benes network and the reverse-exchange network by the following matrices

1) Benes binary network:

$$B = \begin{bmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,2n-2} \\ b_{1,0} & b_{1,1} & \cdots & b_{1,2n-2} \\ \vdots & \vdots & \ddots & \vdots \\ b_{N/2-1,0} & b_{N/2-1,1} & \cdots & b_{N/2-1,2n-2} \end{bmatrix}.$$

2) Reverse-exchange network of the first pass:

$$R_1 = \begin{bmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,n-1} \\ b_{1,0} & b_{1,1} & \cdots & b_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ b_{N/2-1,0} & b_{N/2-1,1} & \cdots & b_{N/2-1,n-1} \end{bmatrix}.$$

3) Reverse-exchange network of the second pass:

$$R_2 = \begin{bmatrix} 0 & a_{0,1} & \cdots & a_{0,n-1} \\ 0 & a_{1,1} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{N/2-1,1} & \cdots & a_{N/2-1,n-1} \end{bmatrix}.$$

Hence, given matrix B , we can immediately obtain matrix R_1 and derive matrix R_2 by performing the following permutation:

$$a_{j,i} = b_{k,n+i-1}, \quad \text{where } j = r_i^{-1}(k) \text{ and } 1 \leq i \leq n-1, \quad (19)$$

VII. UTILIZATION AND LIMITATIONS

Like the other blocking-type multistage interconnection networks, the reverse-exchange network also has its limitations. It cannot do the identity permutation, $+1 \bmod N$, etc., in one pass. But by its properties, we find the way to reconfigure the network to achieve all interconnection functions (including the identity permutation, $+1 \bmod N$, etc.) of other networks in one pass. It also facilitates the two-pass structure which can realize arbitrary permutations. In this section we shall look for impacts of using the reverse-exchange network along with the capabilities of the reconfiguration and the two-pass structure in parallel processing systems.

The bit-reversal permutation is vitally important to the computation of the fast Fourier transformation. The flip network and the shuffle exchange network cannot realize the bit-reverse permutation in one pass. The permutation class of $C_{j,k}^{(n)}$ and $\bar{C}_{j,k}^{(n)}$ which are realizable by the reverse-exchange network in one pass clearly indicates that the scrambled data can be aligned in bit-reverse order and the bit-reverse data can be restored in the original scrambled order.

In the memory-to-processor interconnection organization, the multidimensional access memory [5], [19] can be accessed by words, bit slices, byte-slices, etc. A scramble/unscramble network function is required to scramble the data when it is stored into memory and unscramble the data when it is read from memory. In the course of reconfiguration, the terminal link names (i.e., the memory module names and the processor names) are transformed by proper permutations such as ρ , δ , and others. However, if we modify the scrambling and unscrambling accordingly, we can achieve the same result of the multidimensional access in various configurations. Thus, the reverse-exchange network is not only good for fast Fourier transform (FFT) problems, but is also good for other problems in one pass through the use of reconfiguration. In the processor-to-processor interconnection organization, the output link is fed back to the input link, and the reconfiguration done on terminal links on one side automatically affect the other side. Therefore, the reconfiguration scheme does not seem promising in the processor-to-processor interconnection organization if the implementation cannot remove this restriction. However, the two-pass structure is good for both interconnection organizations.

An array computer can be composed of large numbers of processors for the fast processing of large problems. However, in some circumstances, the computation should be divided into subgroups, and each group, either identical or heterogeneous, can be performed in a small subarray of processors and achieve the efficiency through parallelism. Hence, it is convenient in these cases to be able to partition the computer into various subarrays. In the two-pass structure, the partition can be done for various sizes of subarrays. The permutation which may not be exactly of size N or of 2 's power size is allowed. However, in one-pass, the partition can only be done for an equal size of 2 's power. We have shown the partition definitions $S_{q,k}^{(n)}$ and $\bar{S}_{q,k}^{(n)}$ for the reverse-exchange network. The partition of some other networks can be found in [17].

There is still more work to be done for implementing the reconfiguration and the two-pass structure. It is interesting to know how many configurations are needed to realize arbitrary permutations in one pass. It also seems important to develop computation algorithms in terms of the configurations of a network. To realize a permutation required in an algorithm, we should have the knowledge of how to name the terminal links and how to control the switching elements. Some algorithms may need two or more permutations. A mechanism to work out a compromise configuration should be developed. If we cannot work out a general way, at least we should try to specify the relationships between frequently used permutations and configurations of a network. On the other hand, the routing scheme used in the two-pass structure needs the control algorithm background on the Benes binary network. The looping algorithms need memory storage for computing the control pattern and the computing time needed is in the order of $(N/2) \log_2 (N/2)$. Lenfant's algorithm [8] overcomes those deficits. However, it is restricted on some frequently used permutation families. It is worthwhile to extend his control mechanism to more generalized cases.

VIII. CONCLUSION

The reverse-exchange network is a valuable interconnection network not only because it can be well adapted to some important algorithmic processes such as FFT problems, but also because it serves as an excellent reference from which many fruitful results can be obtained. Previously, it was used to identify isomorphic structures among a class of multistage interconnection networks. In this paper, we first derive the functional relationships among the class of multistage interconnection networks as shown in Table I, using the functional property of the reverse-exchange network. According to the functional relationships, we propose a reconfiguration scheme which can enhance the network capability. Then we specify the admissible permutations through a set of theorems and derive a set of recursive control algorithms to realize some useful permutations. Using the reverse-exchange property, we also prove that the recursive control algorithms actually work. Finally, we prove that arbitrary permutations can be realized on a two-pass structure of the reverse-exchange network. The routing scheme of the two-pass structure is also derived.

The feasibility of reconfiguration presents a fact that a network can accomplish various interconnection functions of

other networks in one pass. From the study of functional relationships, we observe that we can implement other networks on a reference network through permuting names of terminal links of the reference network. The number of permutation patterns needed for such a renaming purpose is surprisingly small. For example, the baseline network needs only two patterns, ρ and δ , to accomplish various interconnection functions (see Table I). The reconfiguration scheme is especially good for the memory-to-processor interconnection organization.

The two-pass structure offers many flexibilities for parallel processing. The structure can realize arbitrary permutations. With this structure, the parallel processing system can be partitioned into subsystems of arbitrary sizes. This kind of system can be used to execute any algorithmic process. The two-pass structure is not only good for the memory-to-processor interconnection organization, but is also good for the processor to processor interconnection organization.

Finally, we like to point out that we do not discount the merits of other types of multistage interconnection networks. On the contrary, these works on the various types of networks mentioned in Table I are useful, and more works are encouraged since any one of these networks can be configured on the reference network topology. Furthermore, any network can serve as the reference network for the reconfiguration and the two-pass structure since they are topologically equivalent. It is our intention to use the reconfiguration and the two-pass structure to remove the limitations of any one of these networks.

REFERENCES

- [1] T. Feng, "Data manipulating functions in parallel processors and their implementations," *IEEE Trans. Comput.*, vol. C-23, pp. 309-318, Mar. 1974.
- [2] H. J. Seigel, "Analysis techniques for SIMD machine interconnection networks and the effects of processor address masks," *IEEE Trans. Comput.*, vol. C-26, pp. 153-161, Feb. 1977.
- [3] K. E. Batcher, "The flip network in STARAN," in *Proc. 1976 Int. Conf. Parallel Processing*, pp. 65-71.
- [4] M. C. Pease, "The indirect binary N -cube microprocessor array," *IEEE Trans. Comput.*, vol. C-26, pp. 548-573, May 1977.
- [5] D. H. Lawrie, "Access and alignment of data in an array processor," *IEEE Trans. Comput.*, vol. C-24, pp. 1145-1155, Dec. 1975.
- [6] L. R. Goke and G. J. Lipovski, "Banyan networks for partitioning multiprocessor systems," in *Proc. 1st Ann. Symp. Comput. Architecture*, Dec. 1973, pp. 21-28.
- [7] T. Lang, "Interconnections between processors and memory modules using the shuffle-exchange network," *IEEE Trans. Comput.*, vol. C-25, pp. 496-503, May 1976.
- [8] J. Lenfant, "Parallel permutations of data: A Benes network control algorithm for frequently used permutations," *IEEE Trans. Comput.*, vol. C-27, pp. 637-647, July 1978.
- [9] S. E. Orcutt, "Implementation of permutations functions in Illiac IV-type computer," *IEEE Trans. Comput.*, vol. C-25, pp. 929-936, Sept. 1976.
- [10] H. S. Stone, "Parallel processing with the perfect shuffle," *IEEE Trans. Comput.*, vol. C-20, pp. 153-161, Feb. 1971.
- [11] C. Wu and T. Feng, "On a class of multistage interconnection networks," *IEEE Trans. Comput.*, vol. C-29, pp. 694-702, Aug. 1980; also see *Proc. 1978 Int. Conf. Parallel Processing*, Aug. 1978, pp. 197-205.
- [12] H. J. Seigel, "The universality of various types of SIMD machine interconnection networks," in *Proc. 4th Annu. Symp. Comput. Architecture*, Mar. 1977, pp. 70-79.
- [13] V. Benes, *Mathematical Theory of Connecting Networks*. New York: Academic, 1965.
- [14] D. Opferman and N. Tsao-Wu, "On a class of rearrangeable switching networks," *Bell Syst. Tech. J.*, vol. 50, pp. 1579-1618, May-June 1971.

- [15] S. Anderson, "The Looping algorithm extended to base 2^t rearrangeable switching networks," *IEEE Trans. Commun.*, vol. COM-25, pp. 1057-1063, Oct. 1977.
- [16] J. Lenfant and S. Tahé, "Permuting data with the Omega network," *RADC Final Rep.*, Nov. 1978.
- [17] H. J. Siegel and S. D. Smith, "Study of multistage SIMD interconnection networks," in *Proc. 5th Annu. Symp. Comput. Architecture*, Apr. 1978, pp. 223-229.
- [18] T. Feng, C. Wu, and D. Agrawal, "A microprocessor-controlled asynchronous circuits switching network," in *Proc. 6th Annu. Symp. Comput. Architecture*, Apr. 1979, pp. 202-215.
- [19] K. E. Batcher, "The multi-dimensional-access memory in STARAN,"

in *Proc. 1975 Sagamore Comput. Conf.*, p. 167; also in *IEEE Trans. Comput.*, vol. C-26, pp. 174-177, Feb. 1977.

Chuan-lin Wu (M'80), for a photograph and biography, see p. 702 of the August 1980 issue of this TRANSACTIONS.

Tse-yun Feng (S'61-M'67-SM'75-F'80), for a photograph and biography, see p. 702 of the August 1980 issue of this TRANSACTIONS.

Concurrent Search and Insertion in AVL Trees

CARLA SCHLATTER ELLIS, MEMBER, IEEE

Abstract—This paper addresses the problem of concurrent access to dynamically balanced binary search trees. Specifically, two solutions for concurrent search and insertion in AVL trees are developed. The first solution is relatively simple and is intended to allow several readers to share nodes with a writer process. The second solution uses the first as a starting point and introduces additional concurrency among writers by applying various parallelization techniques. Simulation results used to evaluate the parallel performance of these algorithms with regard to the amount of concurrency achieved and the parallel overhead incurred are summarized.

Index Terms—Concurrent access, data bases, parallel processing, performance evaluation, search trees.

INTRODUCTION

DYNAMICALLY balanced binary search trees are valuable data structures for implementing symbol tables and directories. This paper deals with the problem of concurrent access to trees built by one of the most widely studied of the balancing techniques, namely, AVL trees. It has been shown [1] that the AVL tree construction is the most efficient method of balancing binary search trees when operations are limited to insertion and searching.

It is not difficult to imagine an application in which concurrent insertion and retrieval of items in a table maintained as an AVL tree would be desirable. For example, a compiler designed to operate in a parallel processing environment might

be organized such that several processes require access to the symbol table. In an earlier paper [2] we considered this problem of parallel compilation and found that sharing the symbol table among the proposed parallel processes presented a major conflict. Therefore, investigating the possibility for concurrency in the manipulation of these data structures is important.

In this paper we present algorithms for concurrent search and insertion in AVL trees. This work is related to similar studies with B -trees [3], [4], [8]–[10] and uses the same basic approach of placing locks on nodes of the tree. Another recent paper [7] discusses concurrent manipulation of binary search trees. Our presentation begins by defining our notation in terms of the AVL insertion algorithm for a sequential environment. Next, we outline the parallelization techniques applied in the design of two solutions for concurrent search and insertion. Finally, simulation results on the performance of these parallel algorithms are summarized.

DEFINITIONS

We assume that the reader is familiar with the terminology and operations associated with binary search trees. An AVL tree is defined to be a binary search tree such that for any node n in the tree,

$$|\text{height}(T_l(n)) - \text{height}(T_r(n))| \leq 1$$

(where $T_l(n)$ and $T_r(n)$ denote
the left and right subtrees of n).

Detailed algorithms for manipulating AVL trees can be found in [6]. However, we will briefly describe the insertion algorithm in order to establish the terminology and because an

Manuscript received August 25, 1979; revised April 20, 1980. This work was supported in part by the National Science Foundation under Grant MCS 76-09839.

The author was with the Department of Computer and Information Science, University of Oregon, Eugene, OR 97403. She is now with the Department of Computer Science, University of Rochester, Rochester, NY 14627.